PI Agent vs Claude Code

engineers. I found the only true clawed code competitor and it's not what you
think. Every single engineer is limited by the tools they use because the tools
you use shape what you believe is possible. So, when's the last time you've asked yourself, "How is my
agentic coding tool limiting me?" If you want to expand beyond what everyone else is doing, you clicked on the right
video. You need new tools to expand what you can do. as the first viable agent
coding tool. Cloud code has changed software engineering. I love cloud code.
I was an early adopter and I bet big on it. If you did too, you've been massively rewarded. But as you know,
there are levels to this game and every tool has limitations. Let's be real for a second. Clawed Code got cancer. What
do I mean by that? Successful products must grow to meet new profit motives.
Remember, this is a for-profit tool. That means doing things that maximizes profit over user satisfaction. With
growth, you'll also tend to serve the masses more and more instead of your
original niche audience. For cloud code, that means us mid to senior level engineers. You can see this happening in
cloud code month after month. Remember anthropic cloud code, it's a private company. They can do whatever they want.
So we can complain or we can put on our engineering hat and do something about it. In this video, I want to share a
tool that engineers are starting to pick up more and more to help you hedge against cloud code and break through the
current limits of your agentic coding tool. This tool is the open-source, unopinionated, customizable
counterattack to claw code. Let's have a slice of pie.
There are many coding agents, but this one is mine. The catchphrase says it
all. Let me show you exactly what I mean by that. Here you can see something incredible. These are all unique
instances of the PI agentic coding tool customized for a specific purpose and a
specific job. One tool, many versions. Let's start with a minimal slice of pie
and work our way up incrementally. So, by the time you finish watching, you'll know exactly how to customize your
agentic coding tool down to the text color. Why am I calling Pi the only
cloud code competitor? How can this be true? How is this a counterattack on cloud code? There are two big themes

you're going to see as we work through this. Open- source and customizable. First, let me give everything away that
we're going to talk about. If you're interested in these ideas, definitely stick around for the video. We're going
to talk about the design philosophy between cloud code and PI. And then I'm going to share three tiers of PI
increasing in capability. We're going to start with the basics of the agent harness, move to agent orchestration,
and then finally move to meta agents. We're going to break down the obvious stuff that you need to know. And then
we'll talk about prompt context engineering tools, hooks, and then the ultimate strategy for how to go about
using these tools, not separately, but together. I think in ans not or the
engineering world is too complex to pick single winners and the ultimate theme here is we are hedging our agentic
engineering. I've written thousands and thousands of prompts and thousands of agents with cloud code. This isn't about
picking one winner. It's about picking the best tool for the job of engineering. So if these ideas interest
you, stick around. Let's jump right in. We first have to talk about the design philosophy between these two tools in
order to understand how they truly differ. So for cloud code, as soon as you open this thing up, you have a great
out ofthe-box default experience for a gent coding. The floor here has gotten
really low. Any engineer and frankly any PM, any designer, any vibe coder can
jump into this tool and start getting value out of it. Now PI agent is very very different. And the author, we'll
talk about the author Mario in a moment here, but his approach here is very unique. If I don't need it, it won't be
built. Okay. So already we have contrasting ideas and the philosophy of how this tool is built. All right. So
let's talk about safety. So cloud code as you know comes with like five out of the box
modes. You have to confirm everything. This is a great start. It's
a great on-ramp to agent coding. But PI agent says uh security in agentic coding
is mostly theater. It's pointless. It's it's stupid. It actually slows down the true value of agents. I completely
agree. There is no safety mode here unless you build one in. But by default, this thing has full access to your
device. And so you can see already PI agent is really leaning toward that engineer that knows what they're doing.

System prompt. This is a big one. The engineers working on cloud code. They've worked hard to create a 10,000 token
prompt. They have a lot of good best practices and opinions baked into the system prompt. And I'm sure you've
wondered this. Is all that really necessary? Uh PI agent doesn't think so. Okay. Pi agent says we have 200 token
prompt. Let's let the model cook. let the model do what it does best, which is think, reason, and solve the problem.
Next, we have observability in cloud code. More and more of this tool has been abstracted away, right? It's like
you don't need to know the exact tools. You don't need to know what's happening right now. Unless you dive into hooks,
unless you really start digging into the engineering of it. Pi agent is a little different. This tool really has nothing to hide. Uh, in fact, it wants to
surface as much to you as possible is as useful. All right. And then here's a banger. As you know, you know, Cloud
Code, again, for-profit company, great company, but it's all about profit. They are prioritizing the cloud models,
right? They want you to use the cloud models, right? Thankfully, they're great. I prefer them over other models.
Pi doesn't care. Pi wants you to use any model you want to to solve the problem. So, this immediately kind of changes the
fabric of your agent coding experience when it's any model versus specific models. All right, so let's just really
really dial into these raw differences. What does this mean as an engineer? Okay, so we have closed source versus
open source. We have the agenda coding leader with unique minimal approach. We have great out of- the-box defaults,
customizable to the core, and you saw that right here, right? As we work through these examples, you're going to
see exactly how you can dial into everything down to the font color. We're going to move all the way up to powerful
multi-agent orchestration, which is insane that you can, you know, customize a tool to that ability. But PI gives you
that capability. We're going to look at that in a second. Do it our way with some customization. We have to give Cloud Code some credit. It is a decently
customizable tool. Pi takes us a step further, right? If you don't like a new feature, disable it, overwrite it, or
pin a version. Once again, open source always gives you that ability. You can just pin and forget. Here is kind of a

where things get tricky, right? With great defaults comes strong opinions. When Cloud Code rolls out a change, you
can't really do anything about it. They're not going to add a feature flag for everything they roll out as much as some of us wish they would. As mentioned
with Pi, you can just roll it back anytime you want. Use our models, disincentivize using others. Pi doesn't
care. You can change anything you want. And last but not least here, we have programmatic support for both. All
right, so this is super important as we talk about on the channel. Your agents must have programmatic support. This is
how you move into outloop agent coding. This is how you build products with agents and this is how you get out of
the terminal. I think with the whole open claw phenomenon, people are starting to understand why that's so important. is so that you don't have to
sit behind the terminal 247 and babysit your agent. So this is the raw difference between these two tools. Now
this summarizes in the fact that PI is really for the advanced engineer looking for that edge of customization and
extensibility. While claw code is the best out ofthe-box agentic coding tool. All right, I think that's well known
now. They are the leader in the space. They created the agent coding space by putting together key ideas, specifically
great architecture, a great agent harness and models powerful enough to drive them. But things have changed.
There are many models that can drive the agent harness now. And the, you know, the secret or the trick of how to build
great Asian harnesses is out.
Let's showcase some of Pi's capabilities. Right? I'm really hyping this tool up. I want to present a balanced approach to the value
proposition of PI because sometimes you are just going to want to reach for that out-of-the-box claude code and get the
job done. There's nothing wrong with that. But let's start with the basics of the PI agent harness. For our first slice of Pi, we're just going to use the

PI v0 Default PI

out ofthebox PI. Okay, so I'm just going to type pi and you can see exactly what
you would expect, right? A terminal interface opens up with a, you know, simple agent loop. We can type ping and
get a response. Here we have the thinking always exposed and we have a nice footer or status line as cloud code

likes to call it. And you can see here that we are using that new claude sonnet 4.6 model with thinking turned on to the
max. Inside of pi you have all the standardized features that you would expect out of a powerful agent coding
tool. You can type slash and you can see all the kind of default commands. We're not going to go over these. These are
all kind of table stakes, right? Fork tree, login, start a new session, compact, resume, blah blah blah. Okay,
so these are all there. Those aren't interesting. That's not what we're going to focus on. What we are going to focus on is how you can take this to the next
level and make this tool your own. Remember the catchphrase here. There are many coding agents, but this one is
mine. Let's go ahead and close this version and let's start customizing our pi. Everyone likes their pi a different
way. So, we can go ahead and copy this one. So, let's say that you're an engineer that likes to stay in the flow,
right? You know the importance of staying in the flow, of staying focused. So we can use pi-e and then we can
enable what's called an extension. So this is how you make pi your own, right? This is how you cook your own pi. Let's
go ahead and copy this, paste it in. And now you can see something really cool. There is no other information on the

PI v1 Pure Focus PI

screen in this terminal except for this bar. Okay. So now same thing ping. And you can see the response. So you know we
don't know the model. We don't know the context. We don't care. I'm in the flow. I don't want to see anything. Okay. So, I just want to show you in a really
blatant example of how customizable this is. You can rip out everything. So, if you want to stay in the flow, you use
this and then it's just you talking with the model. The exact model you're using doesn't matter. Your context doesn't
matter. Nothing matters. All right? Just you and the agent just flowing back and forth. What else can we do with this
tool? Being purely focused in the flow isn't always going to get you a concrete result. You do want some information to
know what model you're running and how much context you have left. So, we can run something like this. And the great part about Pi, notice this, is that I'm
stacking extensions. So I'm now going to put together two extensions in one. Very cool. So I'm going to copy this line,

paste it in, and you can see here we have our own customized footer. Cloudset 4.6. And we have our context window on

PI v2 Minimal PI

the right. Okay. So same deal. Ping ls. We can queue up messages as you would expect, right? LS ls ls ls. You can see
that this is getting queued up and the agent will just keep firing this off. All standard features, right? The thing
I want to showcase here is that you can customize your footer. Okay, fantastic. You can do that in cloud code too.
Nothing special, right? This is where we get to customization. We have this agent here that's going to push our capabilities into new ground. So, this
is something already cla cannot do. So, I'm going to copy this and I'm going to paste it here. And what you're going to

PI v3 Cross Agent PI

see here is the different locations I've picked up my commands, my skills, and my
agents. So, you can see here it's picked these up in multiple locations. Okay. Okay, so we have an extension where we
can set where we want to load the components of your powerful agent decoding tool. So you can see here it's
picking up all of my custom global skills, 41 skills, but also in this directory, you can see we have a
classic/prime command. And you can see I've picked up a bunch of agents as well. And so out of the box, you can
already start to see how we're starting to customize this agent, right? We're deciding where it loads the key pieces
that make up your agentic coding tool. Skills, commands, agents. Let's keep pushing this. So, here's something
interesting. Have you ever run tons and tons of agents and just kind of lost track of what's going on, right? It

PI v4 Purpose Gate PI

takes you time. It takes your context, right? Your personal human engineer context to kind of catch up with what an
agent has even accomplished. This is something that's happened to me over and over. And so, you can build an agent like this with Pi. Right away, as soon
as I boot it up, it's asking me a question. What is the purpose of this agent? So, I'll type explore and
discover this codebase. And I'll hit enter here. And now you're looking at one of the key elements of Pi. This is a

widget. And so a widget is basically a piece of UI that sticks around for the terminal session. Okay. So classic ping
prompt here. Said pong. Nothing new there. But you'll notice that my purpose is still there. And I've done something
special here inside of this extension. And we'll take a quick look at how these TypeScript extensions are actually written later on in the video. But what
I've done here is I've added this purpose. I've appended it to the default 200 token system prompt. And so now my
agent has a little more global steerability. You can see there ready to explore the codebase. It's already starting to lean that way in its
response. You know, we can just fire off a classic prime command. It'll fire up that uh custom slash command and start
understanding the codebase. All right. So very cool. I'm going to stop my agent here and show off another feature. In
fact, I'm not sure if I have this enabled. I'll hold off on showing off this feature. You know, you can kind of
guess what's going to happen here. Um theme cyclinger. You know, you'll notice that this UI looks completely different
from my other agent. So, we'll hold off on that and let's look at another example. Right? So, now we're starting
to stack up extensions. Right? Keep in mind at any point I can just combine these. I'm mix and matching these
different capabilities of my customizable agent via an extension. Remember, there are many coding agents,
but this one is mine. I love that slogan. Okay. Um, so tool counter,
right? Let's just keep customizing what we can do with our fully customizable agent, right? So, let's try this off.

PI v5 Tool Counter PI

And you can see here we have a new footer. So, uh you know, we can type ping. There it is. So, you know, as you
would expect, we have a context window with the model. We have our codebase name with the branch. And you can see
something cool here. Waiting for tools. Okay, interesting. Why is it waiting for tools? Let's type prime. And now, as our
agent runs, it's going to record the exact tools that it's been running. Another just kind of nice feature out of
this tool, you can kind of build in whatever customizations you want to have around your status line or as PI calls
it the footer. You can uh you know really customize it to make it your own. We have the tokens in and we have the tokens out here. So again the footer is

just one location, right? A lot of agent decoding tools they have support for you to modify the footer, but we can do much
better here. This is pi. Let's go to this counter widget and I'm actually going to just add another extension here
to kind of showcase this. I'm going to add the theme cyclinger feature and we'll show that off in just a second. I'm going to copy all this, fire off my

PI v6 Tool Counter Widget PI

customizable agent. And now we have a couple different things going on, right? We have that classic footer and we have
a widget and we can cycle our theme. So, let me just put something on the screen here. Uh, I'll just type ls. I just want
to showcase more of the UI here. Okay, ls no response. So, you can see my key
clicks here. And so, check this out. First off, you can see we have a widget here. Tools 2 and you can see the actual
commands that we ran. If I do Ctrl X, just like you saw there, I can update the theme. I think Cloud Code comes out
of the box with like four themes or something. I created all these themes. All right. So, I have uh you know, you
can see here 13. I'm cycling through 13 customized themes. Of course, my favorite classic synth wave inspired by
synth wave 84. And we can continue to cycle through. So, I'm going to hop all the way back to synth wave, you know, and then we can do things like this,
right? read all TS files. And you know, notice how the capabilities that I'm showcasing, they're not really related
to the model. These are layers on top of the model. And these are layers on top of the context and on top of our
prompts. We're operating on the agent harness. Okay? So, this is a really important theme that I want to communicate to you here is that the
agent harness allows you to kind of uplevel what you can do with your agent coding tool. All right? And we're just
scratching the surface. We're just at the group one example. So you know you can see here our tools are getting tracked. If we had custom tools they
would get called here as well. And let's go ahead and continue. So one of the things that you'll see here is that
unlike cloud code pi does not have sub aent support. Okay. And so there's just
no sub aent support. So we have to build this in. Okay. And this is the kind of cost to using this tool is that
everything has to be built yourself. But um that is also the great part about it right is that this is your agent

decoding tool. Fully customizable open source. It's yours. All right. So, if we copy this, paste it in here. You can see

PI v7 Subagent Widget PI

here that we are running the pure focus extension. So, I just want to be purely focused, but I've also added sub aent
support. All right. So, in a sub aent widget, I can do something like this. I'm going to type /sub and then I'm just
going to type a random prompt. I don't know, you know me, I like to do the classic ping prompt. And so, you can see there a sub agent ran and responded.
Okay. And so, just like before, it is going to persist in the user interface. This is very important. We're hooking
into the life cycle events and persisting UI and updating the UI whenever we need or want to. And our
agent has responded to our primary agent. Look at that. We just added sub agent support to our agent. Now, this is
custom support. So, there are really no limits on this. There are no limits to what you can do. Okay. So, for example, let's go ahead and just hit up. Let's get that command again. Summarize this codebase in one.
We'll do two sentences. And I'm going to fire off multiple agents. So every time we run that command, a new sub agent
spawns. And you can see here these are all unique instances. So imagine you have engineering jobs. Imagine you have
uh you know one-off jobs that you need to fire off. Imagine you have you know testing, reviewing, documenting tasks.
You can just fire up a sub agent in your own way. And you can see above here if I shrink this down a little bit, give us some more space. Our primary agent is
collecting all the results. So sub agent five done, sub agent three done, sub agent two done, so on and so forth. Very
very powerful capability here, right? And I have full control over this. I can build the sub aent system of my dreams
here. And so can you slash sub remove. Let's get rid of sub aent uh 4. There it
is. It's gone. Right. Let's get rid of sub aent one. And now let's continue running sub aent 3. And I want to say
expand into four sentence description. And obviously like these aren't actually useful prompts. I just want to show off
the capabilities of customizing your own PI agent. Okay. So there we go. Sub agent number three. You can see it's on
turn two and it's expanding its response. Okay, so we have our own sub agent system. We build this in a single

extension. Remember I just stacked these others. Uh we also have the theme cyclinger. So I can come in here X and
we can cycle our theme. Drop a like, drop a comment, subscribe. Make sure you're plugged into the signal. I hope
you can see the vision here. It's not just about customizing the prompt or the system prompt or a specific agent,
right? Or a skill, right? That's great. Frankly, that's all table stakes. Now, if you want to expand what you can do,
if you really want to differentiate your agentic engineering and frankly just your engineering, you want to be able to
customize everything surrounding your agentic engineering experience. All right? And so that means having control
of the agent harness. Let me be super clear about this. I love cloud code. I'm going to keep using it. In a moment
here, we'll talk about strategy about how to decide which one to use and when to use each tool. But if you want to go
beyond, you cannot be doing and you cannot be using the tools that everyone else is using. Right? It's called, you
know, the middle of a distribution for a reason, right? Everyone's using cloud code now, okay? Uh that unique value is
is, you know, still there. There's a lot of capability there. We're going to keep covering on the channel. But I also want to start pushing you in some new
directions here for your engineering work. Make sure you like, comment, subscribe so that the algorithm knows you're interested in continuing to push
your limits by pushing the tools you use, right? Because every engineer is limited based on the tools they use. And
so when we're done with our sub agents, we can of course come in here and sub uh clear. And now we're fresh. And we can
of course just type / new. This is a built-in PI tool or PI command. And that will reset the session. Okay. So great
stuff. Uh very very clean, minimalistic UI. You know, you can see here I like being in the flow. I don't want to see
all this extra information. So there's one more here before I want to go back to like comparing these tools together.
All right. Now, this is a big one. So I want to clear out. Going to run this. And you'll see here we have a new extension that I'm calling till done. So

PI v8 Tilldone PI

not to do, not to done. This is till done. Okay. So let me show you what this
thing can do. I'll just type ls. And you'll notice something really interesting here right away. So ls wants me to ls. You can't do an ls. So so I

blocked ls from running. My agent must add a to-do item in the list before it
runs ls. So check this out. It's going to finish up here and let's scroll back up. We have a till done list, not a
to-do list. This is a till done. My agent must work until this work is done. And you can see here I've added my own
custom rules to this. Okay? And so if you're starting to think about, you know, Ralph Wiggum or you're starting to
think about agentic loops and you're starting to think about adding determinism to your agents via hooks or
full-on code that wraps your agents, you're on the right track. That's exactly what we're doing here. I block
my agent using hooks. Of course, PI has a very extensive hook system we'll break
down in a second. But you can see here ls was blocked and it needs to create a task list and add an item to the task
list before doing that. Then it must manage the task, execute on the task and then mark it as done. Okay. And so now
you can see here in my footer, I've updated my footer. I've customized my Genta coding tool to be a tillone agent.
Okay. This thing is going to run till it's done. What do I mean by that? Let me show you a more concrete example. All right, so here's what I'm going to do.
create trees.mmd. And you know what? Actually, what I'm going to do here is I'm going to make our model dumber because I want to show you that
controlling the harness can make up for a lot of model mistakes, right? Even the top tier models. But let's go and dumb
this model down. Uh right now we're running Sonic 4.6. I'm going to go ahead and look for that haiku. An okay model,
but we're definitely talking about bottom tier intelligence. All right, that bottom third. Now, so here's what I
want. Here's my micro task. create tree.mmd that maps out all the files in
this codebase in a tree structure just file names don't read any files right I don't want this to take forever so I'm
going to kick this off and now the till done is going to uh be the primary mode of operation okay and so my agent must
have items in the to-do list and it must complete the item so it's already done this right so create treed with codebase
structure very good right we can now you know load tree.md you can see that this is a net new file I have not committed
this. And we have a great tree. All right. So, fantastic. Let's push this a little further. For each TypeScript file, read it and add a one-s sentence

summary to the tree file. Create a to-do item for each file. So, now I'm going to leverage this to-do list. And you can
see my agent wants to create a new list. In order for it to move on to this next set of tasks, it has to ask me. This is
part of the agentic workflow that I've set up with this extension. So, I'm going to go ahead and hit yes. There we go. And now it's going to create 16
tasks and start working through them piece by piece. And so our Haik coup agent doing a bunch of work, doing it
very quickly, and saving us cash. Keep in mind, this could have been any agent. I could have picked the new GLM5. I
could have picked Minmax 2.5. I could have picked Gemini 3 Flash, right? There are many models that could do this work.
And now you can see it's going to update it. It's read all these files. And now it's going to incrementally toggle all
the items and mark them done. First in progress, and then done. There we go. And now any task that isn't complete,
the agent has to work through and mark complete. And so you can see here that it's it's cutting some corners here. I'm
going to accept this. It's trying to clear the list. Uh any agent that wants to clear the list has to run it by the engineer. All right. So this is a great
tool for inloop agentic coding. So I'm just going to go ahead and said yes. It did the job. And so the tool served its
purpose. So our till done list, you know, got these descriptions added for every single.ts. You can see all there's
all thets files. Fantastic. Another great part about the till done list is that if a task is not complete, right?
Kind of as you saw here, if a task is not complete, the agent is prompted to keep going. So very very powerful tool.
You can see that I'm customizing the agent coding experience, right? The whole thing. And I'm plugging into the
right hooks, right? I'm building out new UIs to aid and drive the experience forward. We had some choice dialogues
along the way. So I'm just painting the picture of how powerful this can be for you. Now that's the tier one kind of basics of PI. Let's go ahead and talk
about the agent harness. Let's compare PI to cloud code. We of course know open source versus closed source. PI agent
comes with just 200 tokens. Cloud code super opinionated. It's got a lot of best practices in there. We have a big
difference in default tools here. If we hop back to an instance here and if I run list tools, I'm going to fire off

this custom slash command. You can see here that it has to create a new till done. Perfect. And then it's going to
update that to in progress. And I was going to write it out to a tools.mmd. So this is just a reusable prompt that I
have and use to get the tools out of the agent. This tool is very limited. You
can see here that our PI agent has just four tools. Read, write, edit, bash.
Okay. And then it has that tool that that custom tool that I created here in task management. So you can see, you
know, the the under workings of that tool here. And just with this, I created my own task management system. All
right. So the customizability is all there but you know notice this just four tools this agent is operating with half
the tools of cloud code out of the box. All right and so once again you know you can see that uh this thing is very bare
minimum. Okay default permissions as mentioned PI runs in YOLO mode out of
the box hooks. Um this is how you tap into the life cycle of the agent as you would expect. Pi has a lot more plug-in
points. Claude has a lot of the essential ones, which is great, but this means you get more control over every
little thing that happens. All right. And then modes, we talked about this, right? There's no real mode in Pi. It's
just yolo mode. It's just do whatever you want. This is where, you know, with PI agent, you really will want to know
what you're doing. That's why I think this is a tool for advanced, you know, mid senior plus level engineers where cloud code lets you really break things
down. It has a lot of opinionated defaults for operating. And you know, if you've been with the channel, you know
that I always operate in bypass permissions mode, which is also known as just YOLO mode. And so, customizability,
once again, is a big deal. Uh, Cloud Code, to be super fair, lets you customize things to a decent degree. But
PI agent just takes it to the next level. You can customize basically everything. Models, uh, this is pretty straightforward, right? You basically
get access to every model when you use PI. We know that, you know, again, Cloud Code is a product. They want you to use
their models. All right? right? They want you to do it their way. PI agent doesn't care. Use whatever you want.
OOTH login, you know, you know that you can use the API key or your Pro or Mac subscription. Uh PI agent lets you use

any one that's available. If there's a way to use your Chatt Plus, GitHub copilot or Gemini CLI plan, PI agent
actually supports that, right? Cost licensing, same kind of deal. We can just kind of run through this. Open source proprietary subscription costs
bring your own. They both has a /cost command. This is exactly what you would expect, right? And this is one of the big problems that people are having with
cloud code. Um, they're really locking down the usage of their plan. You can really only use their plan on cloud code
or the API keys. All right. I'll have engineers DM me about this, like asking for validation on this. I've always said
that they would shut that down and that they would close that. Um, I'm surprised they left the uh O key on the plans open
for so long, but here we are. They finally shut it down. You know, it's all API keys or dedicated plans. Nothing
more to say here. So, this is where things get interesting. Context engineering and prompt engineering. As you know, cloud.md is the default memory
file for cloud code. PI agent uses agents.md and falls back on cloud. All right. Skills both fully support skills.
Skills are industry standard now for agent coding. Everyone supports it. It's all about where you place them for that
quick auto pickup. Commands, same deal. Sub aents. This is where things get interesting. As you saw, cloud code has
native support via the task tool. Pi does not have support. That's why you saw we have to build that in ourself. PI
is a minimal agent decoding agent built for you to customize it. Task list to-dos. Once again, as you saw, we built
that in ourselves. We did it manually, while Cloud Code has the built-in uh task list. Now, it's not called to-do
anymore. Multi- aent capability. We just saw it. We just covered this in a previous video. Cloud Code now has
multi- aent capabilities where you can deploy teams of agents to work together and communicate through a centralized
message bus. PI does not have any of that. All right. Uh but as you saw and you know not to give it away you can
build this yourself here in a PI agent system prompt you can overwrite the system prompt in cloud code. Um with PI
agent you can do the same. So uh this is really a tie but Enthropic doesn't really want to share their system prompt
although it you know it's not super super hard to get but the system prompts are of course customizable. This is a

key part of the core 4 context model prompt tools. If you cannot customize the system prompt you can't use the
tool. All right so this is pretty wellnown. There are no limitations around this. They both have compact features. When you're running out of
context window, nothing new there. All right, programmatic and SDK. So, they both have great support for programmatic
mode. As mentioned, if the tool does not have programmatic mode, throw it out the window right away. You must be able to
build Outloop a Gentic coding tools with the agent harness. Okay, so they both have support for this. The big
difference here is that if you're using Pi, they have a TypeScript SDK that you kind of have to use to get true customization. You can run agents
directly via bash on both sides, but um cloud code wins on the SDK support here. They both have export functionality.
Nothing new there. And we know that on costs you must use the API key for the
cloud code programmatic mode. And that's the exact same with PI agents or uh if the model provider plan supports it, you
can use that. All right. Now, let's move on to our
tier 2 PI agents. Now, this is where things get really interesting. With PI, you can build in multi- aent
orchestration into the experience of the tool. All right, so what do I mean by
that? Let's hop back to our just file and let's move to our second group here. All right, so let me show you an agent
team. All right, so let's reset. Open up a brand new version here. And this is really cool. What you're seeing here is

PI v9 Agent Team PI

an agent team. We have a scout agent, a planner agent, builder, reviewer, documenter, red team. All right. So,
you're probably used to some versions, some number of these agents. Let's kick this off and let me just like show you
how powerful this is. So, this is an arbitrary agent team. And I'll do this scout find all TS files. Instead of my
primary agent doing anything, it has a team of agents that helps them do the work. So, this is another version of
multi- aent orchestration. If we scroll up here, you can see our primary agent didn't do any work. It doesn't need to.
It has agents to do that work on their behalf. Dispatch agent ran. Our scouter
found all the TS files and it returned it to our primary agent. Okay, very very powerful. Let me get out of haiku mode

here. Um get out of the haiku model and let's hop back to that powerful. Let's go and do like a flash, right? Let's do
um Gemini 3 flash here. So we can do something like this, right? We'll have the builder move the tree.md h3s moved
right after the tree file as a bullet point. We're prompting our primary agent
to run the builder agent. But you'll notice here that it used the scouter first to find this stuff. And so it's
running scouter prompts. And then after the scout runs, after it finds the information, it's then going to dispatch
the builder agent to finish the work. There's the builder. The builder is taking the input from the primary agent
that the primary agent got from the scouter. And now it's doing the job, right? It's updating the tree.md file.
You know, you can see here in a very very kind of simple way, we're using multiple agents to drive toward a single
outcome. All right. So now, if we look at our tree, you can see that it's added the description right next to it. You
know, there's a simple version of what multi- aent orchestration looks like. You can see the scouter is running again just to validate that that looks good.
Check this out. Right? We had two agents in this agent team run. And so this is where things get really powerful.
Instead of having a whole set of random agents that you can run, instead of running ad hoc agents, you really want
to be specializing. This is a big theme we talk about on the channel like specialization is the advantage. Now
everyone is going to be running that normal distribution of just junk that comes out of these models. When you give
it your prompt and when you build your own custom agent decoding tool on top of it that's where you really get
differentiated results. Now we can do something else with this. If we type / aent team you can see here we can select
a team. This is a default team that just loaded. If we hit agent teams, you can see here I have different teams of
agents. I'll click on the plan build team. And now you can see I have a planner, I have a builder, and I have a
reviewer. And this all comes from a teams file where you can see I just have a simple YAML configuration for all the
teams that puts together unique combinations of my available agents. So,
you know, remember, let me make this super clear, right? PI does not have sub agent support. So, how am I getting

multi- aent orchestration? I'm building it myself, right? I customize it. This is all coming from this one extension
here. Okay. And of course, I have the theme cyclinger. I'm just stacking that on pretty much on every one of these. So, control X. And so, you can see
there, I'm just cycling through all the themes. My agents are getting customized. Just love the customization here, right? Let's go and look at the
system select. All right. So, let's tone things down a notch from our multi- aent setup. And let's fire this up. So,
system select. What does this do? This allows you to select the system prompt of any one of your agents. As mentioned,
cloud code is a great tool. It has a lot of great defaults, but real engineering work and building solutions for your
users and customers requires being specific. It requires specialization so that you can create unique experiences
and unique outcomes. So, you want to boil that ideology all the way down to your tooling. So, here we have a custom

PI v10 System Select PI

slice of pie where we can type slash uh and then slashsystem. And when I hit enter here, you can see I can now become
any one of my agents. Remember before I mentioned we had that browser agent headless browser support using the
Playright CLI. We covered agent browser automation in our previous video. I'll link that in the description if you're
interested. But now I've updated the system prompt of this tool. Okay. Now I'll just say go to whatever. Let's
let's find something to go to here. Let's go to pi.dev dev summarize the value prop of this tool. This agent is a
browser agent. You can see it fired off the Playright browser skill right away. And now it's running playright via the
player CLI. You can see some of the results there. And there's the response, right? So this agent fully operated a
browser and dug into the capabilities of this agent. Now again, the key part here is that I can change the primary focus
of my agent whenever I want to. All right. And notice what's happening here. You can stack your extensions. Very,
very powerful, very important. These are composable, customizable units of your
own agent. really really powerful stuff here. You can imagine that my real uh
you know primary version of pi I build the extensions in isolation and then I stack up features that I want in
different conditions. I am really against this one agent, one model, you

know, one tool solution to do everything. It's not really how you create real results, right? A lot of
situations require specialization. Let's move on to to damage control. What is damage control? Of course, inside of PI,
you have access to all of the available hooks. In the past, we've talked about building a damage control skill to
prevent your agents from doing dangerous things like this. RM-rf.claude. Let's get rid of claude completely. We
can't because we have hooks blocking it. Just a quick showcase. You can block any command you want. You can control any piece of pie that you want to. All right? And so, we did that with the
damage control extension. Okay? This is where things get really interesting. And if you're still watching, you know, I

PI v11 Damage Control PI

want to thank you by really showcasing how far and how much you can push this. We're going to run this agent chain and
instead of an agent team, check this out. So, I'm going to type chain and let's run a scout workflow. This is a
triple scout recon workflow. So, I have three scout agents chained together.
This is an agent chain, also known as an agent pipeline. Use your team. Look for
all of the commands, skills, and agents list in bullet point and state their
purpose. Check this out. So, I'm going to run a agent chain. This is really
where things go parabolic. I'm specializing everything here. Okay, so I

PI v12 Agent Chain PI

have a three agent execution workflow. It's going to run a scouter, another
scouter, and then another scouter. Okay, so this might seem really simple and kind of like redundant, but you know,
you can put together any agent you want to. And you know, we talk about this on the channel a lot. To scale your impact,
you scale your compute. Imagine you create a plan. And and you know, we don't need to talk about this. We can
actually just do it. Okay. So, I'm going to copy this again. Fire up a new version of this. So, we'll select a
plan, build, review team. And so, you can see we have three unique agents chained together. user team plan out a
new minimal yet unique PI agent. Right away, our primary agent is going to do a
little research to understand what's going on here. Then it's going to kick off its agent team. And so this is very

powerful. We have the orchestrator working on the top level and then it's going to dial its work into its team.
You can imagine what's happening here, right? The planner is going to execute, create a plan for the work. The builder is going to build it and the reviewer is
going to review what was built and make sure that it's right. You know, once again, this is just one of, you know, quite literally infinite agentic
workflows, right? Infinite pipelines, infinite chains of agents that you can build. Okay. And so you can see our
three scouter team is done. If we scroll up here, you can see that it's broken down where our agents are. It's broken
down some of our skills tools. So, it thinks that skills are tools. We didn't properly explain that. And we have
commands and extension hooks, right? Not great. We probably want to throw a smarter model at this problem and give it some more improved setup context. But
the idea here is the same, right? I had three scatter agents run back to backto back over the output of the previous
agent. Okay, agent chain, also known as an agent pipeline. And if you're a tactical agent to coding member, this
probably looks or starting to smell familiar to you. We are a few steps away from having this being an ADW, an AI
developer workflow. I'll leave that concept inside the course. Let's just keep moving here. And we can see that
our planner is finished and our builder is running. Okay, so we're going to let this agent cook. It is running a pipeline of agents, not just an agent
prompting a pipeline. We have an orchestrator agent preparing the prompt and then firing off to the team. And
then you can see the build is finished. Now the reviewer is going to run get status. We can see that it created some
type of totem extension. No idea what that is or if that's going to be valuable. All right, so that's tier two,
right? you can fully orchestrate your agents. Very powerful. We have to give credit where it's due. So, this was
created by Mario Zechner. He's a prolific engineer. I've been following his work for a while. You know, if we go
on to um GitHub here, I sponsored, you know, just three engineers and Mario is
one of them for his great work on Pi. All right. So, big shout out Mario. I think he's shouted out the channel before. Not sure, but he has just
created a great great piece of software. And you might not know this, but this is the agent coding tool that powers

openclaw, previously maltbot, previously clawbot. This tool is very very capable and as you can see it is customizable to
the core. All right, so there are many many reasons to choose this tool over another. Big shout out Mario doing great
work and you know you can see he's been building a lot recently with the release of this great tool. So tools MCP
servers. Let me just quickly go through the kind of highle ideas here. One interesting part about PI kind of
continuing on that trend is that it does not have MCP support. I think Mario realized relatively early that you can
just call the skill via a script or a CLI. All right, so it kind of skipped that step. It skipped that growing phase
that cloud code went through. Tool override. This is another cool capability. You can override the default tools. Edit, write, bash, all of them
can be overwritten. MCP support not there. Tool observability available in both. And of course default tools, we've
covered that. multi- aent orchestration as you saw this is all there cloud code has a lot of this built in but it does
not have Asian chains or Asian pipelines it does have Asian teams pi doesn't have any of it but once again you can build
it and I hope this theme you know I've kind of repeated this theme a million times now you can do anything you want with pi and it solves that like main
problem of the agent harness for you like the the main agent loop is done it's complete and then it just gives you
a full playground to operate on everything else right the best parts of the agent hooks and events. Uh, this is
kind of interesting. They both have a whole set of events. Some of them are in
line, some of them are not. I'm going to leave a ton of resources for you in the description. And one of them will be a
full kind of breakdown of all the hooks and events. And you know, it really comes down to this, right? Like what is
customizable about these tools, right? Let's just be super clear about this, right? This is why PI is so powerful.
You can customize a system prompt. You can use over 25 hooks. You can update the footer, aka the status line. You can
change the themes. You can pick any model you want. You can pass in your own CLI flags. Okay? You can register tools
not just in programmatic mode, but inside of your inloop agent coding tool. All right? You can't do this in cloud

code, right? You have to give it a skill or an MCP. Custom editor mode. Don't really care about that. But if you're a Vimm guy, supports there. Key bindings,
right? You can register custom key bindings. As you saw, we we were cycling through themes. And permission by default is just YOLO mode. This is not
for engineers that do not know what they're doing. All right. Customization. As discussed, cloud code has some
minimal customization, but PI just takes it all to the next level. They both support message queuing. I consider this
a, you know, table stake feature. So, they both have it, which is great.
So, the last agent I want to cover here is really an idea. And the idea is you want to be building agents that build
agents. You want to be working on that meta level so that you can quickly build out your solutions. Okay. So, if we look
for Pi Pi, you can see here that I have a meta agent. And so, I'm going to kick this off. What you'll see is that I have
broken down the capabilities of PI into their own agent expert. So, we're
combining a lot of ideas we've talked about on the channel. You know, we've talked about agent experts. We have an entire lesson on it inside of Agentic
Horizon. Link in the description for that. This is taking multi-agent orchestration to that next powerful
level where you have custom agents that your orchestrator agent is composing and controlling. Let me just showcase uh how
powerful this can be. So I'm just going to say prompt all eight experts. Have them concisely state their purpose. And
so when we fire this off, you can see that my primary agent prompted all eight agents in parallel. And so these are
pre-existing agents that have a precise purpose to explain how to build and
create new PI agents. This is a meta PI agent. And I've delegated all of the
special, you know, unique parts about PI into their own dedicated agents. This is a piece of the future of agentic

PI v13 Meta PI

engineering. One agent is not enough. You want to be stacking these. And once again, here's that theme of
specialization. Okay, these generic agents are generic agents. If you want to survive the SAS onslaught, you better
be building specialized solutions that solve problems better than anyone else can. Again, that boils down to your
tools and the limitations you have around your tools. You can imagine as soon as I started spinning up all of

these agents to create this demo for you, I stopped around agent four or five and I said, "Click where I am. I'm
building agents once again in a different specialized format, right? Building PI agents. Let me create a meta
agent to generate these to spec with the right customizations. Okay. And so that's what I have here. I have built a
PI agent that can query experts, subdomain experts, pull together all the
information it needs, and then the primary agent will actually make the edit. Okay, I'm out of time here. I'm going to link this in the description
for you to dig into yourself. You'll have full access to this codebase. All the kind of juiciness is in extensions
directory. And if we just hop into one of these, you can see that we have, you know, the till done is a, you know, some
700 line TypeScript file that breaks down exactly how this works. And if we just do a quick collapse here, you can
get the gist of it. We registered a new command till done. We registered a tool for the agent, right? So we can call
this whenever it wants to, we have an on input hook that we're catching on. We have another hook on agent end on tool
call session, so on and so forth. Right? You can really see that in order to build these powerful PI agents, you just
want to understand structure and then you can template it for your agents to deliver these results for you on repeat
at scale. But as I mentioned, you know, as you know, a kind of additional theme on the channel, knowing what your agent
is doing is engineering. Okay, not knowing is vibe coding. All right, I want to keep emphasizing that on the
channel. I want to upgrade all the vibe coders. You know, however many vibe coders are watching, shout to you for
really trying to dig deeper and understand things at a deeper level. And for every engineer watching, don't fall into the vibes. Stay sharp with your
engineering. Understand what you're having your agents do. Agentic engineering means knowing what your
agent is doing so that you don't have to look. If you're vibe coding, it means you're not looking and you have no idea
what your agent is doing. Okay? Big big difference between the two. I don't want to sound uh super harsh here. All right?
There are areas where cloud code shines. Where cloud code really shines is the enterprise platform adoption. If you're

a big company, if you're a big team, you have to use cloud code. PI agent is really only experimental, right? Use it
inside of your products in very specific ways. Don't deploy this across your entire enterprise, right? Cloud code has
much better support for that. They're winning on the enterprise level. And this is what you should expect, right? PI agents literally built by one guy in
open source. Let's let's wrap this all up. Who is PI agent for? You probably have the gist by now. If you want full
control over the agent harness, system prop tools, event loop, use PI. If cloud code does what you need and you don't
need more, use cloud code. There's nothing wrong with that. But know that you will be limited by what this tool offers and you'll have to be fine with
some more lock in. So pi is great if you want to use any model out the box. If you want a hedge against cloud code, if
you don't want to get locked in, definitely use pi. Cloud code is great for those out of the box, you know,
simple customizations. If you need to use a stable agent that doesn't break with changes, you know, frankly, both
these these tools change quite a bit. But with Pi, you know, an open source tool, you can just pin the version. You
can fork it, pin it, and it's done. That's the gift of open source. If you want maximum customizability, again, you
want Pi. On the other hand, if you're a big team, if you're an enterprise, you need to use Cloud Code. They have all
the enterprise features. There's no debate here if you're an enterprise. All right? You know, PI's for you if you've outgrown and you've kind of mastered all
the outofthebox features and you want to push agents further. The kind of meta theme I think about all the time is is
that you can't get ahead of the curve by doing what everyone else is doing. The fact of the matter is is that cloud code
is now a mainstream tool for engineers, for PMs, for designers, for all types
now, right? As I mentioned, cloud code has gotten, you know, a curse, the curse of success, the curse of growth almost
in a cancer-like way. Now, it must grow, right? It's a huge profit point for Anthropic. With that, of course, comes a
lot of great things, but it also comes with massive trade-offs. Let's talk strategy here, right? I'm not against cloud code. I still use it. I use it
every single day. But I am starting to hedge against cloud code with a tool like Pi. So, my strategy here is simple.

Bet big on the leader, which is clearly cloud code, but hedge with open source with a highly customizable solution. All
right. There are a lot of open source options out there. I think Pi is the most customizable. Comment down below if
I'm wrong. if you think there's another tool that is more customizable than what you've seen here. We're just kind of
scratching all the ways you can customize and take these ideas further. All right, so that's my strategy, right?
Um, right now it's something like 80% cloud code and then I'm starting to hand off 20% to PI for that deeper level
experimental nextG agent coding. It's nice to have a quick tool where I can rapidly prototype ideas and get ROI for
building out specific agent teams and reusing a great paradigm, you know, a great set of teams. You probably want to
do it a specific way and PI lets you do just that. Zooming all the way out here, I've built a dedicated agent coding tool
that helps me generate new PI agents. Okay, so notice that, right? We've zoomed all the way out. The agent coding
path looks something like this. You start with a base agent. You improve it to a better agent. You become proficient
with your context engineering and your prompt engineering. Then you customize those agents and that's where a lot of
value is. And then you add an orchestrator agent to help you control, manipulate, CRUD all the specialized
agents that you've built, right? Most engineers, you know, most builders are using cloud code for that. Just out of the box cloud code. But you can take
that much further. That orchestrator is very, very powerful. And we're back on that key theme, that kind of edge of
agentic engineering. You want to be orchestrating this, right? The big question is how much can you scale this?
How much can you push, you know, useful agents, specialized agents doing useful work with your orchestrator? Because at
some level, you just can't be in five, 10 terminals, you know, in loop, outloop, web version, local version. Um,
at some level you need something like this to kind of guide that entire experience. That's the strategy. Pi for
control for deep customization and experimental nextgen agent decoding. All right, I think in ands not ors I
recommend you do the same. Use the right tool for the job at the right price. Don't limit yourself. As engineers, we are limited by our tools. So experiment

with this. Give it some time. I have an entire codebase ready for you. Link in the description. Just experiment a
little bit. Play with it a little bit. Just see what you could do. And maybe your engineering mind is going to go off
and it's going to think, okay, I know a problem. I can apply a custom agentic coding tool to that will give us
outsized results. But you'll never find that if you don't just play with it, if you don't give it a shot because you'll
be constantly limited by what you're seeing everyone else do and the normal distribution of tools that everyone's
using, right? Which right now is cloud code. We're in the age of slop. We're in the age of agents. We're going to be experiencing a lot of the same thing
generated by these agents everywhere. Specialization doesn't stop at the model agent or orchestration level. You can
customize your Aenta coding tool. That's the big idea I wanted to connect with you here on today. I'll be sharing some
additional private code bases where we push these ideas to the edge for tactical agentic coding and agentic
horizon members. So, if you're interested, check out my take on how you can scale far beyond the norm with
powerful agentic engineering. The whole goal is to build code bases and run teams of agents so powerful your
codebase runs itself. All right. So, if you're interested in that, if you want to get an additional edge, check that out. Link in the description. I've had
thousands of engineers run through both these courses now. And the value in there is immense. For everyone else,
I'll leave the link in the description for this PI codebase where we break down
all of these different ways to customize the Pi agent. Keep expanding outside the
normal mainstream agent coding experience. There are no winners yet. There are only experimenters, thinkers,
builders and then everyone following them. No matter what, stay focused and
keep building.